Roll No. 2106022

B. Tech. (Computer Engineering) V Semester Examination, 2023

Object Oriented Programming Systems (OOPS)

Time: 3 Hours

Paper No. CEN-504

Max. Marks: 60

Note: Attempt any two parts for Q 2-5. No choice in Q1.

Q1. (a) I. Define and implement a C++ class **Vehicle** with attributes **color**, **licenseNo**, 1 default constructor and destructor, 1 parameterized constructor, 1 copy constructor, static members **count** and **illegalCount** that keep track of the number of total vehicles and the number of unlicensed vehicles (without license no) respectively. You may add logic to the class *with proper explanation*. [2]
CO1

II. Can we pass a **Vehicle** object to the copy constructor *by value*? Explain.

III. Define & implement a class **FakeFactory** that *only* has a static member function **copy** which takes a reference to **Vehicle** and returns a new **Vehicle** object of the same color and blank license number. List the order in which constructors & destructors will be called when the following code is executed:

```
Vehicle realHonda("DL1T90XYZ", "black");
Vehicle fakeHonda = FakeFactory::copy(realHonda, "lemonPink");
```

(b) I. *Object oriented design accords with reality more so than functional design.* Argue for OR against this statement. [2]

II. Explain the use of **static** data members and **static** member functions giving code snippets as examples. Why do we need such constructs? [2]

III. Explain the use of **const** member functions, **const** member function arguments and **const** objects. What kind of restrictions apply for each? [2]

Q2. (a) Define and implement a class called **Circle** that has attribute **radius** and getter/setter functions. Provide overloaded operators for this class, "+" and "<" . The + operator returns a circle whose area is the sum of the two operand circles. The < operator returns **true** if the area of the left operand is strictly less than that of the right operand. Show how these operators will be invoked. [6]
CO2

(b) I. Write a conversion operator *within the class* **Vehicle** (from Question 1(a)) that converts from class **Vehicle** to a class **ElectricVehicle** (assuming it's already [3]

defined). Electric vehicles have the same color and the license number takes on a prefix "**EV-**". [3]

II. Overload the stream insertion operator << for the **Vehicle** class that prints "**<licenseNo>:<color>**". Provide the operator declaration and implementation.

(c) I. Define and implement a class called **TemperatureK** that stores temperatures in degrees Kelvin. Provide the following:

A. A one-argument constructor that takes a **float** value. [2]

B. A conversion operator that converts to primitive type **float**. [2]

II. State whether each line of code in the following will run without errors and which of the above (constructor, operator, both) will be used:. [2]

```
float fltTemp;
TemperatureK temp1 = 300;
fltTemp = static_cast<float>(temp1);
TemperatureK temp2(102);
fltTemp = temp2;
temp2 = fltTemp;
```

Q3. (a) CO3 I. What are *access specifiers* in C++? How are they used to control access to class members? What is their role in inheritance? [3]

II. Explain *abstract classes*, *virtual functions*, *pure virtual functions* in C++. How do these enable run-time polymorphism? Explain using a simple example. [3]

(b) I. Define and implement an abstract class **Shape** with a protected attribute **name**, a public member function called **print** and a public pure virtual function **area**, a 1-argument constructor that takes the name as parameter. Define and implement a publicly derived class **Rectangle** with attributes **length** and **width** and a 3-argument constructor which takes both dimensions and name as parameters. Pass the name up to the base class constructor using an initializer list. Override **print** and **area** with reasonable functionality. [3]

II. Define and implement a class **Square** publicly derived from **Rectangle** with attributes **side** and **name** and a 2-argument constructor with the side and name as parameters. Pass the **side** and **name** to the **Rectangle** class constructor in the initializer list. Override **print** and **area** with reasonable functionality. What is the output of the following code? [3]

```
Shape *myShape = new Rectangle(4, 2, "Rectangle");
Rectangle * myRectangle = new Square(5, "Square");
myShape->area();
myShape->print();
myRectangle->area();
myRectangle->print();
```

(c) I. Differentiate between *late* (or *dynamic*) *binding* versus *early* (or *static*) binding giving at least two differences. [2]

II. If there is a class **Base** with public member function **func1** and a publicly derived class **Derv** with member function **func1**. How can we access the **func1** of **Base** within the **func1** of **Derv**? [2]

III. If **Base** has a protected static data member **data1,** is it inherited into **Derv**? How can we access it within **func1** of **Derv.** [2]

Q4. CO4 (a) I. What is the use of **this** and **super** keywords in Java? Illustrate with an example. [3]

II. Explain the terms *dangling pointer error*, *memory leak* and *garbage collection* in Java with code examples. [3]

(b) Define a public interface called **Crushable** that has a single function called **crush**. Define a public class **Circle** with attribute **radius** that implements **Crushable**. The function **crush** should return an array of 4 **Circle** objects of random radius such that the total area equals the area of the original **Circle**. (Hints: Circle circleArray[] = new Circle[Array_Length]; import java.util.Random; int rand_int1 = rand.nextInt(10); //Generates integer between 0 and 9) [6]

(c) List and explain 3 features of Java that makes it a *completely* object oriented language. [6]

Q5. CO5 (a) What are two *different* ways of achieving multithreading in Java? Illustrate with example code. [6]

(b) I. Using a simple class called **ThrowDemo**, show how to catch a division by zero exception using **try** and **catch** blocks. [2]

II. In the catch block, print out the description of the exception using **System.out.println**. Why does this work? [2]

III. *All exceptions in Java are objects*. Explain this statement and show two levels (root plus first level) of the class hierarchy of exceptions with a diagram. Now add **ArithmeticException** to this hierarchy in the correct place. [2]

(c) Define & implement a class **PriorityThread** that extends **Thread**. In the **run** function simply print "Running". Add a **main** function that runs 3 threads and sets their priorities to 1,2, 3 and then gets and prints the priorities of all 3 and the main thread. [6]